



I AM STILL ALIVE

Soaring Like A Stork[®]

TECHNICAL ASSESSMENT

Software Engineer — SDE 2

Experienced Hire · Web (Django) or App (Flutter)

Position	Software Engineer (SDE 2)
Experience	3–4 years
Engagement	Remote · Full-Time
Assessment Window	10 Days (flexible — see Section 3)
Effort Expectation	~10–14 focused hours
Tracks	Choose ONE: Web or App
Submission	careers@iamstillalive.com

Prepared for the named candidate. Please don't share or publish.

1. Hello, and what this is

Thanks for applying to the Software Engineer (SDE 2) role at I Am Still Alive (IASA). The response on this role was very large — you have been shortlisted to this technical round.

We have designed this carefully. You are a working professional, your time is valuable, and a take-home that drags on for two weeks is disrespectful. So we have kept the scope deliberately tight in size — but high in difficulty. What we are testing is judgement, not stamina. This is a hard assignment on purpose. We pay well for this role, and we expect the bar to reflect that.

This assignment is your technical round.

There is no separate DSA test, no coding-platform exam, no second round of LeetCode. What you submit here, plus the live discussion where you defend it, is what we use to decide. We are hiring an engineer we will trust with code that real cancer patients depend on — so we care far more about how you reason than how many features you stack up. A small system, designed and defended properly, beats a large one built on autopilot.

About IASA and the VTB

IASA is a pre-launch oncology patient support platform aimed at the US market. The clinical heart of the product is the Virtual Tumor Board (VTB).

The basic flow: a cancer patient (a Cancer Warrior) submits a health question. A trained Moderator turns it into a structured clinical case. A panel of Doctors discusses the case in a threaded, comment-and-reply style — not a flat chat. Doctors can choose to participate anonymously. Once enough input is gathered, the Moderator compiles a single verified answer back to the Warrior.

Five roles touch this: Cancer Warrior, Doctor, Moderator, Admin, and Corporate. Because the domain is clinical, correctness, identity handling, and data integrity are not extras — they are the product.

2. About using AI tools — read this carefully

We will say this plainly, because it shapes how we built this assignment:

You are ENCOURAGED to use AI tools. We expect it.

Use Claude, ChatGPT, Cursor, Copilot, whatever you like. At IASA, AI-assisted engineering is how we work every day, and we want to hire someone who is genuinely excellent at directing these tools. Being fast and effective with AI is a skill we value — not something we penalise.

But here is the point: this assignment is built so that AI alone will only get you a mediocre, shallow submission. AI is excellent at the easy 70% — the boilerplate, the CRUD, the obvious patterns. It is weak at the hard 30% — the genuine trade-offs, the conflicting constraints, the domain judgement,

and explaining WHY you rejected the obvious approach. That hard 30% is exactly what we grade. A strong engineer uses AI to fly through the easy part and spends their real thinking on the hard part. That is what we are looking for.

Two things follow from this:

- Because you have AI help, “I ran out of time” is not an excuse we will accept, and our quality expectations are correspondingly high. Use the tools to raise your ceiling.
- You must fully understand and be able to defend every line you submit — including the lines an AI wrote. In the live walkthrough we will ask you to explain, change, and extend your own code in real time. If you cannot, that is disqualifying. This is the real test of whether you direct the tool or the tool directs you.

3. How this assessment works

1. Two tracks: Track A (Web, Django + DRF) and Track B (Mobile, Flutter). Complete only ONE — whichever is your real strength.
2. The brief is deliberately under-specified and contains genuine trade-offs with no clean “correct” answer. Part of what we assess is the calls you make where the brief leaves room — and how you justify them.
3. You deliver three things: the code, a written DESIGN DOCUMENT, and a recorded walkthrough. All three carry weight. The design document is where senior judgement shows — it is not an afterthought.
4. Shortlisted candidates attend a 45-minute live discussion: you share your screen, defend specific decisions, make a small live change, and do one short live AI-pairing exercise with us watching.

Timeline: 10 calendar days from receipt. Plan for roughly 10–14 focused hours across evenings and a weekend. If your week is heavy, email us before the deadline and we will gladly extend 3–4 days — asking is viewed positively, not negatively.

4. Track A — Web Backend (Django + DRF)

Design and build a backend that powers a slice of the Virtual Tumor Board. This is a real piece of what we build — not a textbook exercise. The base feature set is intentionally buildable; the difficulty lives in the constraints that interact with each other.

The scenario

A Cancer Warrior submits a health question. A Moderator converts it into a structured clinical case and invites a panel of Doctors. Doctors discuss in a threaded structure (comments, replies, replies-to-replies). Doctors may post anonymously. When ready, the Moderator publishes one verified answer to the Warrior. The Warrior never sees in-progress discussion — only the final published answer.

Core requirements (the buildable part — AI will help you here)

- Django + DRF, JWT auth, role-based authorization for Cancer Warrior, Doctor, and Moderator.
- A data model for cases, the original question, the structured case, threaded comments, and the published answer.
- An enforced state machine: SUBMITTED → IN_REVIEW → UNDER_DISCUSSION → ANSWERED, plus a CLOSED/REJECTED path. Invalid transitions must be impossible at the API layer.
- Role-correct access on every endpoint, structured error responses, correct HTTP semantics, PostgreSQL, clean migrations.
- Automated tests covering the state machine and the authorization rules.

The hard part — where we actually grade you

These are the constraints that interact and conflict. AI will give you generic answers; we want YOUR reasoning. Address each in code where appropriate, and in depth in your design document:

- ◆ Concurrency. Two Moderators act on the same case simultaneously — one publishes the answer while the other moves it back to UNDER_DISCUSSION. How do you guarantee the case can never end up in a corrupt or contradictory state? Name your exact mechanism and its cost.
- ◆ The anonymity reveal problem. A Doctor posts anonymously, others reply to that anonymous comment, and a third Doctor quotes it in their own reply. The original Doctor now wants to reveal themselves on that one comment. How do you reveal their identity WITHOUT retroactively exposing them in the quotes and replies where they expected to stay anonymous? There is no perfect answer — show us your reasoning and exactly what you trade off.
- ◆ Anonymity vs. audit. Anonymity is only at the presentation layer; the system must always know the true author for clinical accountability. Implement “hidden to peers, known to system” so that it cannot leak through API responses, error messages, list ordering, timestamps, or sequential IDs. Tell us where the leaks could realistically happen.



- ◆ Auditability without bloat. Every clinically meaningful change (who edited, who transitioned, who published) must be reconstructable later. Design this so it does NOT add a heavy write on every ordinary request. Justify where you draw the line.
- ◆ Edit-after-publish. Once a case is ANSWERED, the discussion and the answer become part of a clinical record. What exactly becomes immutable, what stays editable, and how do you enforce it at the data layer rather than trusting the UI?
- ◆ Real-time, honestly scoped. Doctors expect new comments to appear without refresh. Describe (don't fully build) a Django Channels + Redis design, and be specific about what breaks first when 50 doctors are active on one case at once.

Bonus (only if you have spare time)

- OpenAPI/Swagger docs or a Postman collection.
- Dockerfile / docker-compose for clean review.
- Rate limiting or basic observability hooks.

5. Track B — Mobile App (Flutter)

Design and build a Flutter module for the VTB experience, mainly from the Cancer Warrior's side with a secondary Moderator view. The base UI is intentionally buildable; the difficulty lives in correctness under bad conditions and in protecting a frightened user's trust.

The scenario

A Warrior submits a question, tracks their cases through changing statuses, and reads the final verified answer when published. A Moderator view (same app, role-switched) shows a queue of cases to move forward. The app must behave correctly on a flaky mobile network: never lose a submission, and never show an in-progress answer as if it were final.

Core requirements (the buildable part — AI will help you here)

- Flutter (Dart) with Provider, clean separation of UI / state / data layers.
- Case List with status indicators, Case Detail, a validated Submit Question flow, and a Warrior/Moderator role switch.
- Explicit loading, empty, error, and success states everywhere — no silent failures, no infinite spinners.
- Local persistence (Floor/sqlite) so cases and any unsent submission survive a restart.
- A mock data source or simple mock API is fine; structure the data layer as if a real API plugs in later.
- Widget/unit tests for your state logic and one critical flow.

The hard part — where we actually grade you

These conflict with each other. AI gives textbook answers; we want YOUR reasoning. Address each in code where appropriate, and in depth in your design document:

- ◆ The offline-submit race. A Warrior submits with no signal, the app retries in the background, but the user re-opens the app and submits again thinking it failed — and meanwhile the first one actually went through. How do you guarantee exactly-once from the user's point of view? Name your mechanism (idempotency keys? local dedupe? something else?) and its failure modes.
- ◆ Stale-answer integrity. The cached copy says a case is ANSWERED with answer X. The server has since revised it. The device is offline. What do you show, and how do you make sure the Warrior is never misled by a stale-but-confident local copy? This is a clinical trust problem, not just a caching problem.
- ◆ Optimistic vs. honest UI. Showing an action as “done” before the server confirms feels fast, but for a cancer patient a status that flips back is frightening. Where do you allow optimistic updates and where do you refuse them? Defend the line you draw.



- ◆ State scalability. As the case list and threads grow, how do you keep Provider state manageable and avoid unnecessary rebuilds? Be specific about your structure, not just “use selectors.”
- ◆ Notification correctness. Describe FCM integration so a Warrior is notified when their case is answered. What happens if the push arrives while offline, while the app is open mid-submit, or after the answer was already revised? Walk through each.

Bonus (only if you have spare time)

- ▶ Calm, healthcare-appropriate UI; accessibility (font sizing, contrast).
- ▶ Theming and a small reusable widget library.
- ▶ A live demo of offline→online recovery in your walkthrough.

6. The Design Document (the part that matters most)

This is the single most important artifact, and the hardest to fake with AI. A generic, AI-flavoured design doc and one written by someone who actually understood the problem are easy to tell apart — we read a lot of these.

Submit a concise document (roughly 2–4 pages, Markdown in your repo or a PDF) covering:

- Architecture overview — how the system is structured and why.
- Data model and state machine, with the reasoning behind them. Rough diagrams help.
- Your answers to the hard questions for your track — in depth, with the trade-offs spelled out.
- Key decisions and the alternatives you REJECTED. For each major choice, what else you considered and why you didn't pick it. This section carries the most weight.
- Security and data-integrity thinking for a clinical product. Not full HIPAA compliance — but show you are thinking about it.
- Self-critique: name the TWO weakest parts of your own submission and how you would fix them with more time. (We weight honest self-critique highly — you cannot critique code you did not understand.)

A required short section: “How I used AI on this”

In half a page, tell us honestly how you used AI tools: what you delegated to them, where they were wrong or unhelpful, where you overrode them, and what you had to figure out yourself. We are not checking whether you used AI — we assume you did. We are checking whether you DIRECT the tool with judgement or just accept what it produces. A candidate who can clearly describe where the AI was wrong scores higher than one who claims they didn't need it.

Write in your own voice. Generic AI boilerplate — “leveraging best practices for scalable, robust solutions”, lists with no real opinion — is obvious and scores poorly. “Here is what I chose, here is why, and here is what I gave up” beats “this follows industry standards” every time.

7. How we will grade you

Scored against the criteria below. Notice the weights — judgement and reasoning together far outweigh raw feature count. That is deliberate, and it is what separates a strong AI-assisted engineer from someone who just pasted prompts.

Criterion	Weight	What earns a high score
Design & judgement on the hard questions	30%	Real reasoning on the conflicting constraints; sound trade-offs; rejected alternatives explained.
Correctness & edge cases	20%	State machine and authorization are airtight; the hard cases are handled, not ignored.
Code quality & structure	15%	Clean, idiomatic, maintainable; good separation; code another engineer can pick up.
Domain & data-integrity judgement	15%	Treats this as a clinical product — trust, integrity, anonymity, auditability respected.
AI-direction & self-critique	10%	Clear, honest account of how AI was used, where it was wrong, and what you overrode.
Testing & communication	10%	Meaningful tests; clear README and design doc; confident, honest live defense.

The live walkthrough — including a live AI-pairing exercise

In the 45-minute discussion we will: (1) ask you to justify specific decisions; (2) propose a hypothetical change (“what if a case could have two Moderators with different permissions?”) and discuss how your design adapts; (3) ask you to make one small live modification to your own code; and (4) give you a small new sub-task to do live, using your AI tools, while we watch how you prompt, judge, and correct the output. Point 4 is not a trap — it is the clearest way for us to see the exact skill we are hiring for. Be ready to explain any line you submit, including the lines AI wrote.

8. How to submit

1. Push your code to a PRIVATE GitHub/GitLab repo (grant access to the reviewer email we share on request), or a public repo if you prefer.
2. Include your README and your DESIGN DOCUMENT (with the “How I used AI” section) in the repository.
3. Record a 5–8 minute walkthrough: show it working, but mostly explain your key decisions and the hard-question trade-offs. Put the link in your README.
4. Email the repo link to careers@iamstillalive.com with the exact subject line below.

Email subject line (copy exactly):

SDE2 Assessment — [Your Full Name] — [Track A: Web / Track B: App]

In the email body, include:

- Full name, phone number, and current notice period



- Total years of relevant experience and current role
- Which track you chose, and the repository link
- Walkthrough video link

README must include

- What you built and which track.
- Exact setup/run steps on a clean machine, and the command to run the tests.
- Links to your design document and walkthrough video.
- Any assumptions you made.

9. Ground rules

- AI tools are encouraged. But you must fully understand and defend every line you submit — including AI-written lines. Inability to explain or extend your own code in the walkthrough is disqualifying.
- The design decisions and the reasoning must be genuinely yours — you will defend them live, and do a live AI-pairing task.
- Don't share this assignment or your solution with other candidates. We run similarity checks; matches usually mean both submissions are dropped.
- Use realistic but entirely fake data. Never use real patient information, even for testing.
- Where the brief is ambiguous, make a sensible decision, document it, and proceed. Good judgement under ambiguity is exactly what we are testing.

We are genuinely looking forward to seeing how you think. Good luck.

— Team IASA